

Master Technical Blueprint & Operational Design Specification

A highly granular, industrial-grade operational blueprint detailing live telemetry architectures, multi-channel automated messaging flows, transaction checkout variations, role segregation protocols, and the deployment roadmap for native client scaling.

AUTHOR & SYSTEMS ARCHITECT

Dunsimi

DOCUMENT VERSION

v2.0 (Official Stakeholder Release)

SECURITY TIER & CLASSIFICATION

Strictly Confidential // Board View Only

TARGET PLATFORM MILESTONE

Responsive Web App to Native Mobile

1. Strategic Scaling & Multi-Phase Deployment

To maximize core product efficiency, mitigate initial mobile runtime compilation bottlenecks, and optimize resource deployment, the Fast X Logistics Platform is strictly structured as a three-stage progressive scaling model. This ensures a unified technical foundation that matures in lockstep with transaction velocity and operational scaling.

Phase 1: Advanced Progressive Responsive Web Application (Launch State)

The system will launch as a highly optimized, responsive Progressive Web Application (PWA) constructed using semantic HTML5, modern modular CSS3, and standard client-side JS/TypeScript runtimes.

Justification: This approach eliminates customer onboarding friction by bypassing the app-store installation requirement. Business vendors can execute bulk bookings instantly from their desktops, and freelance riders can operate the portal directly from standard mobile browsers. In-memory and browser storage layers are implemented to bypass aggressive mobile browser process throttling.

Phase 2: Hybrid Shell Wrappers & Native Bridge Integration (Transition State)

As courier density and active contracts expand, the core codebase will be packaged into hybrid wrapper shells (e.g., Capacitor, Cordova, or Flutter Webview bridges) to establish an immediate presence in the Google Play and Apple App Store.

Justification: This allows the platform to retain a unified codebase while accessing native operating system channels, enabling background execution locks, native device hardware geofencing libraries, and direct OS-level push notifications to courier devices.

Phase 3: Fully Native / Cross-Platform Mobile Applications (Enterprise Scale)

At peak operating scale (exceeding 10,000 daily orders), client and courier workspaces will migrate to dedicated native applications (Swift for iOS, Kotlin for Android, or optimized React Native/Flutter modules).

Justification: This unlocks native OS thread execution, background hardware-level GPS polling, and hardware-wake locks that are completely immune to browser tab freezes, providing the foundation for enterprise-level logistics coordination.

2. Live Location Tracking & Telemetry Loop

Maintaining real-time visual tracking on mobile browsers is technically challenging because mobile operating systems aggressively suspend background JavaScript execution to preserve battery life and data. Fast X bypasses these restrictions through a Hybrid Telemetry Engine.

2.1 Throttled Buffering & Geofenced Polling (The Rider App)

Continuous GPS streaming over standard mobile browsers drains device batteries and consumes excessive data. To resolve this, Fast X implements throttled coordinate buffering via `navigator.geolocation.watchPosition()`.

State Transition Rule: Coordinates are captured and queued only if the rider crosses a 50-meter vector threshold since the last upload OR if exactly 30 seconds have passed. This reduces network data payload weights by up to 85%.

2.2 Resilient Client Cache Schema (Offline Dead-Zones)

When riders navigate traffic dead-zones or switch to background apps (e.g., WhatsApp, phone calls), browser network streams drop. Fast X integrates an automatic local buffering schema using browser `localStorage` or `IndexedDB`.

Rider coordinates are timestamped and buffered locally on the device during drops. The moment the browser detects active network connection recovery, the app dumps the entire cache in a single consolidated batch write transaction to the server.

2.3 High-Throughput Memory Telemetry Cache (The Backend)

Writing continuous coordinate streams directly to primary SQL relational tables (e.g., PostgreSQL Orders table) creates significant transaction bottlenecks. Instead, telemetry coordinates are routed directly to an in-memory cache layer (e.g., Redis Key-Value stores mapping `rider_id` -> `{lat, lng, timestamp}`).

When customers poll for tracking updates, they query the fast in-memory cache. Once the order transitions to DELIVERED, the temporary coordinate stream is wiped completely, leaving only pickup time, delivery time, and total calculated distance in the PostgreSQL historical logs to prevent storage bloat.

2.4 Client-Side Vector Path Interpolation (The Customer UI)

To avoid the high monthly fees of live map routing APIs (e.g., Google Maps, Mapbox Routing), Fast X utilizes client-side linear path interpolation. The customer's browser loads the open-source Leaflet map and interpolates coordinates. If the rider's coordinate changes from Point A to Point B, standard frontend vector math smoothly animates the vehicle icon over a 5-second slide, providing a seamless visual tracking experience.

3. Dynamic Booking & Validation Subsystems

To ensure operational security, prevent logistical mismatch errors, and optimize transaction rates, the booking system is designed as a strict, stateful validation funnel.

3.1 Dynamic LGA-to-LGA Pricing & Geographical Typing

To eliminate external address autocomplete lookup costs, Fast X uses a cascading geographical selection matrix. Senders select a pickup State, which unlocks the corresponding LGAs/Hubs under that state. The Dropoff selection operates in the same manner. This structural data input maps directly to an internal database pricing fee matrix (LGA A -> LGA B), returning a precise, instant base quote to the sender without API overhead.

3.2 Size Categorization & Insurance Underwriting

To prevent disputes at pickup, bookings strictly require size selection presets (e.g., Document/Flyer, Small/TEMU Box up to 2kg, Medium Box up to 5kg). Senders must declare the parcel description and financial valuation. The system automatically appends a dynamic insurance risk premium (1% of declared value) as a corporate liability safeguard.

3.3 High-Volume Multi-Parcel Arrays

E-commerce vendors often dispatch multiple packages to different destinations in a single session. Fast X designs its relational tables to map a single Booking node to an array of nested Parcel Waybill entities. Vendors enter their sender details once, then rapidly output separate recipient detail cards, producing distinct tracking waybills under a single consolidated payment invoice.

4. Financial Integrations & Checkout Variations

Fast X implements three distinct payment checkout flows to cater to diverse client preferences in regional logistics:

4.1 Paystack Direct Checkout Integration

The automated payment path. Upon completing booking steps, the client pays on the Paystack inline gateway. The backend listens for asynchronous payment success webhooks, transitioning the order state from PENDING_PAYMENT to PAID_UNASSIGNED.

Safeguard: Webhook endpoints enforce strict Idempotency Locks using a composite unique key constraint of order_id + event_type to prevent double waybill or duplicate rider dispatches from retried gateway notifications.

4.2 Deep-Linked WhatsApp Transfer Checkout

To capture clients preferring manual bank transfers, selecting 'WhatsApp Checkout' places the order in PENDING_WHATSAPP_VERIFICATION. The web app generates a deep-link redirecting the user to the Admin support chat with a pre-filled template message containing their unique Order ID and payment instructions. The dispatcher verifies the physical transfer, locates the Order ID on the Admin Dashboard, and toggles the manual confirmation button, shifting the state to PAID_UNASSIGNED.

4.3 Cash on Delivery (COD) & Rider Settlement Escrow

For e-commerce vendors offering Cash on Delivery, the order is flagged with `Collection_Type: CASH`, bypassing online gateways. Upon delivery, the rider collects physical cash from the recipient. The system immediately registers this cash balance as an active debt in the rider's virtual wallet ledger. To protect company funds, the rider is locked from accepting new jobs until they clear this balance with the system administration.

5. Failure Cascades & Resiliency Workflows

A robust technical architecture must handle operational exceptions gracefully before they disrupt daily logistics loops.

5.1 Telemetry Drop-out Daemon & Heartbeats

If a rider's device goes offline, switches off, or loses cellular connection while an order is IN_TRANSIT, the backend system's background heartbeat daemons detect the anomaly. If no telemetry coordinate is updated in the Redis cache for more than 15 consecutive minutes, the daemon flags the Order ID and escalates it to the active Admin operations board for manual customer support intervention.

5.2 Courier-Side Item Mismatch Disputes

If a client hands the courier a package exceeding the booked size category (e.g., a large microwave booked as a Document), the courier selects 'Reject: Item Mismatch' on their console. This action immediately halts the contract lifecycle, registers a dispute ticket, and shifts the state to ADMIN_REVIEW. The admin can then adjust the booking dimensions, invoice the sender for the pricing difference, or cancel the order.

5.3 Two-Way Verification PIN Handshake

To eliminate delivery fraud, the platform enforces a strict two-way PIN validation finite state machine:

- Pickup Stage: When the courier arrives at pickup, they must request the 4-digit Pickup PIN from the sender. The courier app UI remains locked until this PIN is entered and validated, transition the state to IN_TRANSIT.
- Delivery Stage: Upon arrival at the drop-off landmark, the courier app prompts for the recipient's 4-digit Delivery PIN. Entering this PIN transitions the order to DELIVERED, closing the contract and releasing the courier's earnings from escrow into their virtual wallet.

6. Automated Multi-Channel Messaging Pipeline

Fast X utilizes an automated event-driven messaging queue (via Meta WhatsApp Cloud API or SMS gateways) to keep all participants informed without manual administrative actions.

6.1 Recipient Inbound Hooks

The moment an order transitions to PAID_UNASSIGNED, a database hook fires a transactional message to the recipient's WhatsApp number. The message contains a customized greeting, the vendor's brand name, their tracking link, and the recipient's secure 4-digit Delivery PIN.

6.2 Rider Dispatch Broadcast Hooks

When an order is marked as PAID_UNASSIGNED, the system publishes a dispatch notification packet. This is broadcast to the open courier pool in the corresponding LGA, displaying pickup and dropoff hubs, dimensional specifications, and net payout earnings, encouraging rapid, atomic job claims.

7. User Portals, Dashboards & Permissions

The system enforces strict Role-Based Access Control (RBAC) across public and private portal interfaces.

7.1 Public Interface & Casual Client Console

The public landing pages feature a marketing overview, a login-free global tracking bar, and the dynamic quote estimator. Casual senders access their transaction histories via passwordless OTP logins sent directly via WhatsApp or SMS, eliminating password database management overhead.

7.2 B2B Vendor Workstation

High-volume e-commerce sellers log into a workspace equipped for rapid logistics operations:

- CSV/Excel Uploader: Drag-and-drop parsing engine that converts spreadsheet rows into validated waybills.
- Active Pipeline Tracker: A dense Kanban interface dividing active orders across logical swimlanes.
- Dual-Balance COD Escrow Vault: Separates settled withdrawable funds from pending in-transit cash collections, offering transparent financial bookkeeping.

7.3 Rider Compliance & Execution Interface

Riders access a mobile Progressive Web App (PWA) workspace designed for standard smartphones:

- Onboarding Funnel: Stateful three-step submission capturing Legal Profile, Bank Verification (via Paystack API validation), and Government ID, Address, and Guarantor documents.
- Marketplace & Execution Console: Available Job Feed with atomic locking, and active execution screens featuring the Pickup and Delivery PIN handshake loops.
- Wallet Ledger: A simple dashboard displaying cashout balances, withdrawal triggers, and transaction streams.

7.4 Operational Admin vs. Super Admin Dashboards

To protect company funds and operational continuity, the administrative interface enforces strict RBAC splits:

- Operations Staff (Admins): Manage active dispatches, execute override courier assignments, check pending courier document onboarding folders, and verify manual WhatsApp bank transfer receipts.
- Executive Governance (Super Admins): Modify the global matrix pricing matrix, adjust platform commission variables, provision or revoke staff credentials, and execute manual wallet corrections.

Audit Security Safeguard: Every single administrative action (state toggle, billing adjustment, staff provision) is automatically recorded in a write-once, append-only System Activity Log database, containing timestamps, User IDs, action types, and IP addresses.